

TP Technologie du Web : Javascript

Boutounas Noam : étudiant de L3 Informatique

Module: Technologie du web



Enseignant et date de remise : R.Tomczak pour le 30 septembre (maximum)



Sommaire :

1) Introduction :

Contexte :

Ce TP consiste à travailler principalement en JavaScript afin d'apprendre certaines compétences en manipulation du DOM, gestion des événements et interaction avec des formulaires HTML.

Objectifs principaux :

- Créer un convertisseur de devises (euros, dollars US et australiens) avec deux versions : une avec bouton de conversion, l'autre avec mise à jour en temps réel.
- Manipuler des éléments HTML via JavaScript (querySelector, getElementById, gestion des attributs et propriétés CSS).
- Traiter des interactions utilisateur (clic, saisie, changement de valeur) et modifier dynamiquement le contenu ou le comportement de la page.
- Gérer l'affichage conditionnel de sections via une interface à cases à cocher.

- Utiliser des boucles, des événements et des sélecteurs pour répondre à des consignes précises.

Technologies et outils utilisés :

- **HTML** : structure de base fournie.
- **CSS** : pour la gestion de l’affichage (display, visibility).
- **JavaScript** pas de bibliothèques externes ; utilisation intensive du DOM API.

2) Fonctionnalités implémentées :

2.1) Convertisseur de devise (1ère version) :

Une page web permet de convertir l’euro en dollar américain et australien. Dans cet exemple, l’appui sur le bouton Convertir effectue la conversion comme ce-ci .

Convertisseur		
	Euros	120
	Dollar Américain	121.20
	Dollar Australien	176.40

Explication :

1. Créer la fonction de conversion :

J’ai fait une fonction convertir() en JavaScript. Elle permet de :

- lire la valeur entrée en euros via document.getElementById('euros').value
- multiplier par un taux de conversion (ici 1.01 pour USD et 1.47 pour AUD, par exemple)
- afficher le résultat dans les champs USD et AUD avec toFixed(2) pour limiter à 2 décimales.

2. Lier le bouton :

J’ai ajouté onclick="convertir()" au bouton, comme ça lorsqu’on clique, ça lance la fonction et met à jour les résultats.

Les fonctions importantes ici sont :

- `document.getElementById()` pour récupérer les valeurs des inputs.
- `parseFloat()` pour transformer une string en nombre.
- `toFixed(2)` pour formater le résultat.

2.1.2 Seconde version (automatique) :

Dans cette version, dès qu'une valeur est modifiée (Euros, Dollar Américain ou Dollar Australien), la conversion est recalculée automatiquement.

Convertisseur		
	Euros	144
	Dollar Américain	145,44
	Dollar Australien	211,68

Explication :

Trois fonctions de conversion :

- `fromEuros()` : si on change la valeur en euros :ça calcule USD et AUD.
- `fromUSD()` : si on modifie le dollar américain :ça recalcule les euros et l'australien.
- `fromAUD()` : pareil mais à partir du dollar australien.

2. Les taux de change :

Deux variables (`tauxUSD = 1.01` et `tauxAUD = 1.47`). Ça permet de convertir facilement dans les deux sens.

3. Les événements input :

Avec `addEventListener('input', ...)`. Pour être plus précis, dès que l'on tape ou modifie un chiffre dans une case, la fonction correspondante s'exécute automatiquement et ça met à jour les autres cases.

Les fonctions importantes :

- `parseFloat()` → transforme la valeur du champ en nombre.
- `toFixed(2)` → affiche avec 2 décimales (genre 10.00).
- `addEventListener('input', ...)` → réagit directement quand tu changes un champ.

2.2)Formulaires

2.2.1) Première page HTML

Lien et images

[La page principale de Wikipedia](#)



Des éléments !

☒ Choix N°1 ☐ Choix N°2 ☐ Choix N°3

Volume
0

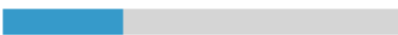
☐ Une case à cocher

Date : 

Barres de progression

Téléchargement en cours :



Espace disponible : 

Explication :

Cette fois-ci nous sommes encore sur une page

HTML/Javascript assez classique.

Pour les titre nous avons utilisé les <h2> et <h3> afin de séparer les différentes parties comme «lien et images» , «barre de progression»...

Les formulaires : on trouve plusieurs types de champs :

- text = écrire du texte,

- radio = faire un choix unique,
- password = cacher ce qu'on tape avec des points,
- range = curseur pour régler une valeur (ex: volume),
- checkbox = case à cocher,
- date = choisir une date,
- button = bouton cliquable.
- Les barres de progression : <progress> affiche une barre qui montre l'avancement d'une tâche. On règle avec value (valeur actuelle) et max (valeur totale).

2.2.2) Modifiez le lien wikipedia pour qu'il point vers le site français

Lien et images

[La page principale de Wikipédia \(FR\)](#)



 Wiki Loves Monuments : photographiez un monument historique, aidez Wikipédia et gagnez !

En apprendre plus

✕

Bienvenue sur Wikipédia

🌐 353 langues ↕

[Accueil](#) [Discussion](#)

[Lire](#) [Voir le texte source](#) [Voir l'historique](#) [Outils](#) ↕



Wikipédia

L'encyclopédie libre que vous pouvez améliorer

[Version pour appareil mobile](#)

[Accueil de la communauté](#)
[Comment contribuer ?](#)

[Portails thématiques](#)
[Principes fondateurs](#)

[Sommaire de l'aide](#)
[Poser une question](#)

Explication :

```
// Sélectionne le premier lien <a> de la page
let lienWiki = document.querySelector("a");

// Change l'adresse du lien vers Wikipédia en français
lienWiki.href = "https://fr.wikipedia.org";

// Changer le texte du lien
lienWiki.textContent = "La page principale de Wikipédia (FR)";
```

ici «document.querySelector(«”a”)» permet de sélectionner le premier lien trouvé dans la page <a>

Ensuite le «.href» sert à modifier l’adresse de lien

et enfin le «.textContent» change le texte affiché à l’écran pour savoir sur quoi nous nous dirigeons.

2.2.3) Lors de l'appui sur le bouton , il faut vérifier que le texte dans l'étiquette soit «Oui» ou «Non». Dans le cas contraire , il faut que le texte de l'étiquette soit «Il faut mettre Oui ou Non»

Des éléments !

Il faut mettre Oui ou Non

☒ Choix N°1 ☐ Choix N°2 ☐ Choix N°3

Volume
0

☒ Une case à cocher

Date :

```
// Sélectionne le bouton "Ok" et le premier input text
let boutonOk = document.querySelector('input[type="button"]');
let inputText = document.querySelector('input[type="text"]');

// Ajoute un écouteur d'événement sur le bouton
boutonOk.addEventListener('click', function() {
  // Récupère la valeur de l'input text et la convertit en minuscules (pour être insensible à la casse)
  let texte = inputText.value.trim().toLowerCase();

  // Vérifie si le texte est "oui" ou "non"
  if (texte === "oui" || texte === "non") {
    // Si c'est bon, on ne fait rien ou on peut afficher un message de confirmation
    console.log("Texte valide : " + texte);
  } else {
    // Si ce n'est pas "oui" ou "non", on change le texte de l'input
    inputText.value = "Il faut mettre Oui ou Non";
  }
});
```

Explication :

Sélection des éléments : On sélectionne le bouton "Ok" et le premier input text

Écouteur d'événement : On ajoute un écouteur qui se déclenche au clic sur le bouton

Vérification : On récupère le texte, on le nettoie (trim) et on le met en minuscules

Condition : Si le texte est "oui" OU "non" alors tout va bien

Action : Si le texte n'est pas valide alors on change la valeur de l'input

ici `input[type="button"]` : Sélectionne l'input dont l'attribut type est "button" (notre bouton "Ok")

- `input[type="text"]` : Sélectionne le premier input dont l'attribut type est "text" (notre champ de texte)

Par la suite on ajoute l'écouteur d'événement avec :

boutonOk.addEventListener('click', function()

Pour continuer on récupère le texte saisi dans l'input avec «inputText.value» , on supprime les espaces avec «.trim()» puis on convertit tout en minuscule avec «.toLowerCase()»

Enfin on vérifie les conditions et on effectue des actions selon le résultat.

2.2.4) En utilisant nextSibling (cf cours), modifiez Choix N°1, Choix N°2 et Choix N°3 par respectivement HP, Casque , Bluetooth. Ajoutez des id à l'HTML si nécessaire

Des éléments !

☐ HP ☐ Casque ☐ Bluetooth

Volume

0

☐ Une case à cocher

Date :

Ok

On commence par ajouter les id dans la partie HTML :

```
<div class="box">
  <h3>Des éléments !</h3>
  <input type="text" value="Premiers pas avec Javascript"><br><br>
  <input type="radio" name="choix" id="radio1"> Choix N°1
  <input type="radio" name="choix" id="radio2"> Choix N°2
  <input type="radio" name="choix" id="radio3"> Choix N°3 <br><br>
</div>
```

Ensuite on modifie les labels des radios avec `getElementById` et `nextSibling`

```
// Modification des labels des radios avec getElementById et nextSibling

// Pour le premier radio (HP)
let radio1 = document.getElementById("radio1");
let label1 = radio1.nextSibling;
label1.textContent = " HP";

// Pour le deuxième radio (Casque)
let radio2 = document.getElementById("radio2");
let label2 = radio2.nextSibling;
label2.textContent = " Casque";

// Pour le troisième radio (Bluetooth)
let radio3 = document.getElementById("radio3");
let label3 = radio3.nextSibling;
label3.textContent = " Bluetooth";
```

`getElementById()` va permettre de sélectionner un élément par son id
puis le `nextSibling` va servir de «propriété» qui va donner le prochain nœud frère dans le DOM.

Ici le `<input>` est un nœud et le texte «Choix 1» qui suit est son `nextSibling`

2.2.5) Au choix Dès qu'un choix est réalisé, modifiez en dessous « Volume » en « Volume HP », Volume Casque» ou Volume Bluetooth». Aide : - utilisez `input[name="choix"]` - utilisez `this.value == "1"` - `nextSibling.nextSibling` Modifiez le maximum du volume de 11 à 100. Affichez la valeur max dans la console.

Dès que cette barre est modifiée, sa valeur s'affiche juste en dessous. Aide : utilisez `this.value` et comme événement `OnInput (input)`.

Des éléments !

Premiers pas avec Javascr

☐ HP ☐ Casque ☒ Bluetooth

.....

 Volume Bluetooth

Valeur actuelle : 91

Dès qu'un choix est réalisé , modifiez en dessous «volume» en «Volume HP , Casque ou Bluetooth» :

```
// Modifier le texte Volume selon le choix radio
let radios = document.querySelectorAll('input[name="choix"]');

radios.forEach((radio) => {
  radio.addEventListener('change', function() {
    if (this.checked) {
      if (this.value == "1") volumeText.textContent = " Volume HP";
      else if (this.value == "2") volumeText.textContent = " Volume Casque";
      else if (this.value == "3") volumeText.textContent = " Volume Bluetooth";

      // Mettre à jour aussi l'affichage de la valeur
      valeurVolumeDisplay.textContent = "Valeur actuelle : " + volumeRange.value;
    }
  });
});
```

Ici la première ligne de code sélectionne tous les boutons radios qui ont `name="choix"`

ça a pour résultat : 'radios' qui contient les 3 radios (HP , Casque , Bluetooth)

Ensuite `if (this.value == "1") volumeText.textContent = " Volume HP";`

avec le `this` qui nous donne la radio sur laquelle on a cliqué

le `this.value` nous donne la valeur de la radio (1,2,3)

et ensuite on compare pour vérifier si c'est la première , deuxième ou troisième avec les `'=='`

Puis après il nous faut plusieurs `«nextSibling»` pour «sauter» les éléments entre la radio et le texte «Volume !»

Grâce à ça lorsqu'on clique sur HP : Volume devient → «Volume HP»
et pareil pour les deux autres.

Modifiez le maximum du volume de 11 à 100 et affichez la valeur max dans la console :

```
// Modifier le maximum du volume de 11 à 100
let volumeRange = document.querySelector('input[type="range"]');
volumeRange.max = 100;
console.log("Valeur max du volume : " + volumeRange.max);
```

ici on va avoir `document.querySelector()` qui est une méthode qui cherche un élément dans la page

puis le `'input[type="range"]'` qui est considéré comme un sélecteur CSS qui signifie que l'élément `input` dont l'attribut `type` est égal à `range`.

Résultat , le `volumeRange` contient la barre de volume de la page. Enfin on le définit sur 100.

Le `console.log` sert de fonction pour afficher un message dans la console du navigateur en concaténant la valeur actuelle de `max` (100) avec `+volumeRange.max`

Dès que cette barre est modifiée, sa valeur s'affiche juste en dessous :

```
// 3. Créer un élément pour afficher la valeur actuelle
let valeurVolumeDisplay = document.createElement("div");
valeurVolumeDisplay.textContent = "Valeur actuelle : " + volumeRange.value;

// Placer l'affichage de la valeur APRÈS le texte "Volume"
volumeText.parentNode.insertBefore(valeurVolumeDisplay, volumeText.nextSibling);

// Afficher la valeur quand on modifie le volume
volumeRange.addEventListener('input', function() {
    valeurVolumeDisplay.textContent = "Valeur actuelle : " + this.value;
});
```

On commence par créer l’affichage avec la valeur `VolumeDisplay = document.createElement("div");` , concrètement ça crée un nouveau bloc vide dans la page.

Ensuite on place ce bloc juste après le texte «Volume»

Et enfin avec les deux dernières lignes de code : à chaque mouvement de la barre , ça met à jour l’affichage avec la nouvelle valeur.

Pour être concis ,on créé un élément qui s'affiche sous la barre de volume, et j'utilise l'événement 'input' avec 'this.value' pour mettre à jour la valeur en temps réel à chaque mouvement."

2.2.6) Modifiez «Une case à cocher» en «Mute»

Aide :utilisez querySelector et label[for='ouinon'] si besoin :

Volume

Valeur actuelle : 50

☐

```
// MODIFIER "Une case à cocher" en "Mute"  
let checkbox = document.querySelector('input[type="checkbox"]');  
let texteCoche = checkbox.nextSibling;  
texteCoche.textContent = " Mute";
```

Ici on a utilisé `querySelector` pour sélectionner la checkbox, puis `nextSibling` pour accéder au texte qui la suit directement dans le code HTML. En modifiant la propriété `textContent` de ce texte, j'ai remplacé « Une case à cocher » par « Mute ».

On commence par sélectionner la case à cocher en cherchant un élément `input` de type `checkbox`. Ensuite avec `nextSibling` on rattrape le texte «Une case à cocher» qui se trouvait après la checkbox dans le HTML
Enfin , on modifie le texte par «Mute»

2.2.7) Un appui sur la case à cocher à droite de « Mute », désactive ou réactive le volume. Aide : utilisez les attributs `disabled` et `checked`

Des éléments !

Premiers pas avec Javascri

☐ HP ☐ Casque ☒ Bluetooth

.....

Volume Bluetooth

 **MUTED - Valeur : 81**

.....

0

☒ Mute

ici on utilise à nouveau un écouteur d'événement 'change' sur la checkbox et lorsque l'utilisateur clique dessus :

this.checked permet de savoir si la case est cochée ou décochée

volumeRange.disabled = this.checked désactive la barre de volume si c'est cochée et la réactive si décochée.

On a aussi ajouté des styles visuels pour que le changement soit plus visible qu'à l'origine

Quand la case Mute est cochée , la barre de volume devient grisée et inutilisable , accompagnée de ‘MUTED’ en rouge. Lorsque la case est décochée , la barre et le volume redeviennent fonctionnelles.

```
// 6. MUTE SYSTEM !
let checkbox = document.querySelector('input[type="checkbox"]');
// Renommer la checkbox
checkbox.nextSibling.textContent = " Mute";

// Désactiver le volume
checkbox.addEventListener('change', function() {
    volumeRange.disabled = this.checked;

    if (this.checked) {
        // STYLE MUTE
        volumeRange.style.background = "linear-gradient(90deg, #ff6b6b, #556270)";
        volumeRange.style.border = "3px solid #ff6b6b";
        volumeRange.style.opacity = "0.8";
        valeurVolumeDisplay.textContent = "🔊 MUTED - Valeur : " + volumeRange.value;
        valeurVolumeDisplay.style.color = "red";
    } else {
        // STYLE ACTIF
        volumeRange.style.background = "linear-gradient(90deg, #4ecdc4, #556270)";
        volumeRange.style.border = "3px solid #556270";
        volumeRange.style.opacity = "1";
        valeurVolumeDisplay.textContent = "🔊 Volume : " + volumeRange.value;
        valeurVolumeDisplay.style.color = "green";
    }
});
```

2.2.8) Ajoutez à la fin de la section Lien et images une image. (src avec par exemple

https://upload.wikimedia.org/wikipedia/commons/b/bd/UPHF_logo.svg),

la largeur (200), rechercher l'élément div Lien et images puis insérer un nœud avec appendChild

Lien et images

[La page principale de Wikipédia \(FR\)](#)



Ici pour

ajouter une image dans la section 'Lien et Images' on a utilisé :

```
// === AJOUT D'IMAGE DANS LA SECTION "LIEN ET IMAGES" ===  
  
// 1. Trouver la div "Lien et images"  
let divLienImages = document.querySelector('.box h3').parentElement;  
  
// 2. Créer la nouvelle image  
let nouvelleImage = document.createElement('img');  
nouvelleImage.src = 'https://upload.wikimedia.org/wikipedia/commons/b/bd/UPHF_logo.svg';  
nouvelleImage.alt = 'Logo UPHF';  
nouvelleImage.width = 200;  
nouvelleImage.style.marginTop = '10px';  
  
// 3. Ajouter l'image à la fin de la div  
divLienImages.appendChild(nouvelleImage);
```

La première ligne permet de sélectionner le titre «Lien Et Images» puis de remonter à sa div parente avec .parentElement

Les 4 prochaines lignes servent à la création globale de l'image. Notamment avec 'createElement('img') pour créer une balise image , ensuite on met en place l'attribution des attributs 'src' , 'alt' et 'width'

Et enfin divLienImages.appendChild(nouvelleImage); qui permet d'ajouter

dynamiquement un élément dans le DOM sans modifier le HTML.
En partie grâce à `appendChild()` pour ajouter l'image à la fin de la section.

3.Réponses aux questions

- Quel est l'événement qui se déclenche lorsque la page est chargée ?

C'est l'événement `DOMContentLoaded`, il se déclenche lorsque le HTML est complètement chargé et parsé

```
// 1. Événement quand la page est chargée
window.addEventListener('DOMContentLoaded', function() {
```

- Quel est l'élément sur lequel se déclenche cette événement ?

Il y a plusieurs réponses possibles : `window` ou `document`

En effet `window` représente la fenêtre du navigateur et `document` représente le document HTML

```
// Les deux fonctionnent :
window.addEventListener('DOMContentLoaded', function() { })
// OU
document.addEventListener('DOMContentLoaded', function() {
```

- Comment désactiver l'affichage

(https://www.w3schools.com/css/css_display_visibility.asp)

On utilise `display: none`
grâce à ça l'élément disparaît complètement

`visibility: hidden` est invisible mais garde sa place

-Comment utiliser une boucle for

On commence par initialiser le compteur à 0 (`let i =0`)

On met ensuite une condition : tant que `i` est inférieur au nombre d'éléments

`[i < elements.lenght]`

on incrémente `i` de 1 à chaque tour (`i++`) puis on accède à sa position (`elements[i]`)

Code Final pour tout cacher sauf le menu :

```
// === CACHER TOUT AU DÉMARRAGE SAUF LE MENU ===

// 1. Événement quand la page est chargée
window.addEventListener('DOMContentLoaded', function() {

    // 2. Cacher toutes les sections .box
    let toutesLesSections = document.querySelectorAll('.box');

    // 3. Boucle for pour cacher chaque section
    for (let i = 0; i < toutesLesSections.length; i++) {
        toutesLesSections[i].style.display = 'none';
    }

    // 4. Garder visible seulement la première section (menu)
    if (toutesLesSections.length > 0) {
        toutesLesSections[0].style.display = 'block';
    }
});
```

C'est grâce à ce menu, que l'utilisateur va choisir la ou les partie(s) il voudrait afficher. Coder ce comportement en JS. :

Menu :

- ☒ Lien et images
- ☒ Des éléments !
- ☐ Barres de progression

Lien et images

[La page principale de Wikipédia \(FR\)](#)



Explication :

1. "Au chargement de la page :"
 - On crée un menu avec des checkboxs pour chaque section
 - Je cache toutes les sections sauf la première
2. "Quand on clique sur une checkbox :"
 - `this.checked` nous dit si elle est cochée ou décochée
 - `sections[index]` correspond à la section liée à la checkbox
 - Cochée → on affiche la section (`display: block`)
 - Décochée → on cache la section (`display: none`)

3. "Résultat :"

L'utilisateur peut choisir quelles sections afficher en cochant/décochant les cases du menu.

```
// SYSTÈME DE MENU
window.addEventListener('DOMContentLoaded', function() {
  let sections = document.querySelectorAll('.box');

  // 1. Créer le menu dynamiquement
  let menuHTML = `
    <div class="menu-options" style="margin-bottom: 20px; padding: 10px; background: #f5f5f5;">
      <strong>Menu :</strong><br>
      <input type="checkbox" id="menu1" checked> Lien et images<br>
      <input type="checkbox" id="menu2"> Des éléments !<br>
      <input type="checkbox" id="menu3"> Barres de progression<br>
    </div>
  `;

  sections[0].insertAdjacentHTML('afterbegin', menuHTML);

  // 2. Cacher les sections sauf la première
  for (let i = 1; i < sections.length; i++) {
    sections[i].style.display = 'none';
  }

  // 3. Gérer les checkboxes
  let checkboxes = document.querySelectorAll('.menu-options input[type="checkbox"]');

  checkboxes.forEach(function(checkbox, index) {
    checkbox.addEventListener('change', function() {
      sections[index].style.display = this.checked ? 'block' : 'none';
    });
  });
});
```

Si vous rechargez la page mais que vous avez coché précédemment des cases sont toujours présentes. Modifiez le callback du démarrage afin qu'aucun choix soit sélectionné :

Quand on recharge la page, le navigateur garde en mémoire l'état des cases radio et des cases à cocher. Résultat : si on avait coché quelque chose avant, ça reste coché au rechargement.

Pour corriger ça, on a ajouté du JavaScript au démarrage de la page (window.addEventListener('DOMContentLoaded', ...)).

Dans ce callback, on a :

1. Récupéré toutes les cases radio avec `querySelectorAll('input[type="radio"]')`.
2. Récupéré toutes les cases à cocher avec `querySelectorAll('input[type="checkbox"]')`.
3. Mis `checked = false` sur chacune pour forcer qu'elles soient décochées.

Résultat : au rechargement, toutes les cases sont vidées, et aucun choix n'est sélectionné.

```
// RESET des cases radio et checkboxes à l'ouverture
let radios = document.querySelectorAll('input[type="radio"]');
radios.forEach(radio => radio.checked = false);

let checkboxes = document.querySelectorAll('input[type="checkbox"]:not(.menu-options input)');
checkboxes.forEach(checkbox => checkbox.checked = false);
});
```

Récupérer l'année choisie de : <input type="date" et l'afficher dans la console. Les deux barres de progression partent de zéro et progressent de 5% toutes les secondes

```
// === RÉCUPÉRER L'ANNÉE DE LA DATE ===
let inputDate = document.querySelector('input[type="date"]');

inputDate.addEventListener('change', function() {
    let dateChoisie = new Date(this.value);
    let annee = dateChoisie.getFullYear(); // extrait l'année
    console.log("Année choisie :", annee);
});

// === BARRES DE PROGRESSION QUI AVANCENT AUTOMATIQUEMENT ===
let progressBars = document.querySelectorAll("progress");

// Initialiser les barres à 0
progressBars.forEach(bar => bar.value = 0);

// Faire progresser de 5% toutes les secondes
setInterval(function() {
    progressBars.forEach(bar => {
        if (bar.value < bar.max) {
            bar.value += 5;
        }
    });
}, 1000);
```

Explication :

Pour la date :

- On sélectionne l'<input type="date">.
- On écoute l'événement change.
- Avec new Date(this.value) on récupère la date, puis .getFullYear() donne l'année.
- On affiche dans la console avec console.log.

Pour les barres de progression :

- On sélectionne toutes les balises <progress>.
- On met leur valeur à 0 au départ.
- Avec setInterval(...), toutes les secondes on ajoute +5 à chaque barre.
- Elles montent donc petit à petit jusqu'à atteindre 100%.

4) Difficultés rencontrées et solutions

Gestion de l'ordre d'exécution du JavaScript

Problème : La première difficulté majeure a concerné l'ordre d'exécution des scripts. Plusieurs portions de code modifiaient les mêmes éléments DOM, créant des conflits. Par exemple, le système de menu cachait des sections que d'autres fonctions tentaient de modifier ensuite. Principalement sur le moment de cocher ou décocher les cases automatiquement à l'exécution

Solution : structurer le code en respectant une logique séquentielle :

- Initialisation des variables d'abord
- Gestion des événements ensuite
- Modification du DOM à la fin

Conflits entre les différentes fonctionnalités

Problème : L'intégration successive des fonctionnalités a créé des interférences. Par exemple, le système de volume et le système de menu tentaient tous deux de modifier les mêmes éléments DOM, entraînant des comportements inattendus.

Après l'ajout du système de menu, la barre de volume disparaissait parce que la section "Des éléments !" était cachée au démarrage.

Solution : Il a fallu réorganiser l'architecture complète du code pour établir une hiérarchie claire entre les fonctionnalités.

Gestion des événements et du DOM

L'une des autres difficultés a été de bien comprendre comment manipuler les éléments HTML avec JavaScript sans modifier directement le fichier HTML. Par exemple, pour changer l'adresse du lien Wikipédia, il fallait utiliser `querySelector` et modifier ses attributs en JavaScript. La solution a été de tester plusieurs sélecteurs (`querySelector`, `querySelectorAll`, `getElementById`) afin de comprendre leur comportement.

La manipulation des textes et des nœuds

Quand il a fallu renommer les boutons radio (par exemple en “HP”, “Casque”, “Bluetooth”), je me suis rendu compte qu’utiliser `nextSibling.textContent` ne donnait pas toujours le résultat attendu, car les nœuds texte peuvent contenir des espaces ou des retours à la ligne. Cela a créé quelques erreurs d’affichage. La solution a été de tester dans la console du navigateur ce que renvoyait chaque sélecteur, et d’adapter le code en ciblant précisément le nœud souhaité. Cela a permis mieux comprendre la structure du DOM

5) Commentaires et suggestions :

Globalement, ce TP a bien fonctionné et m’a permis de progresser de manière concrète dans la manipulation du DOM avec JavaScript. La manière dont sont agencées les exercices, de manière progressives est intéressante : on part de modifications simples (changer un lien, modifier des labels) pour aller vers des manipulations plus dynamiques (conversion automatique, barres de progression animées). Ce côté progressif m’a beaucoup aidé à assimiler les notions étape par étape. Cela permet également de se voir avancer et progresser, par conséquent on a encore plus envie de «se donner» pour conclure l’exercice

Sur le plan technique, ce TP m’a appris à utiliser :

- `querySelector` et `querySelectorAll` pour cibler des éléments précis dans le DOM,
- les événements comme `input`, `change` et `click`,
- les fonctions de base de JavaScript comme `setInterval` pour gérer le temps,
- et des méthodes simples pour manipuler les attributs (`.href`, `.value`, `.checked`, etc.).

6) Auto-évaluation - Évaluer honnêtement votre travail (planification, qualité du code, rigueur de la rédaction, respect des consignes). - Fixer des objectifs réalistes pour les prochains projets

En réalisant ce TP, je pense avoir respecté globalement les consignes et fourni un travail cohérent. J'ai réussi à suivre les différentes étapes demandées et à implémenter les fonctionnalités en JavaScript. La planification s'est faite de manière progressive : je testais chaque partie avant de passer à la suivante, ce qui m'a permis d'éviter de gros blocages. Le code est fonctionnel et relativement clair, même si certaines parties pourraient encore être optimisées.

En ce qui concerne la rédaction, j'ai essayé de rester rigoureux en expliquant les choix techniques et en détaillant les étapes de manière compréhensible pour une lecture aérée et agréable. Toutefois, je pense que je peux encore m'améliorer dans la mise en forme et la compréhension des consignes du rendu pour rendre le compte rendu encore plus lisible.

Pour les prochains projets, mes objectifs sont :

- Améliorer la clarté et la structure de mon code (ajout de commentaires, meilleure indentation, éviter les répétitions inutiles).
- Planifier davantage mon temps pour ne pas me retrouver à finir certaines parties à la dernière minute.
- Approfondir mes connaissances en JavaScript moderne (manipulation avancée du DOM, modules) afin d'être plus efficace et de coder de manière plus propre.
- Développer l'habitude d'écrire un compte rendu plus visuel, en intégrant des captures d'écran et en variant les présentations.

7) Code et GitHub

Pour ce Github , qui est un de mes premiers , j'ai opté pour une configuration plutôt classique :

- Un README
- Un dossier src contenant : le code HTML et Javascript
- Un dossier images contenant : un screen du rendu final du projet
- Un dossier docs contenant : le compte rendu ici présent du TP

Lien vers le dépôt github :

<https://github.com/IIZUNNO/TP1-Javascript-et-Manipulation-DOM>